FormalLLM — Reproducing and Hardening Refine4LLM: Full Project Report

Oct 5, 2026 · @Prakrititz Borah

Executive summary

Starting from a working Z3-based square-root refiner, this phase added independent Coq and Lean certification, a checked HumanEval specification set, and a re-engineered v8 search; it ended with 100/100 square-root successes, 440/440 on the other benchmark problems, and a HumanEval pilot that derives the simple problems every time but not loops or searches.

- **Certification:** every finished derivation can be re-checked by Coq (Rocq 9.1) and Lean 4; Lean certified 21 of 21 obligations of the classic programs, Coq 9 of 21.
- **Datasets:** 20 HumanEval specifications, each checked against the reference solution on every enumerable test input and rejecting 590 of 590 deliberately wrong answers; hand derivations accepted for 17.
- **Baselines:** two open models alone pass 156/147 (gpt-oss:120b) and 154/147 (qwen3-coder-next) of HumanEval/EvalPlus, above the paper's GPT-4 (145/128), close to Refine4LLM (150/150).
- **Refinement on HumanEval:** 121 valid runs, 53 derived a program, 41 pass every test and all 41 are Coq-certified, covering 7 of the 20 problems.
- **Findings that go beyond the paper:** the model was shown `2 * (n / 2)` as `2 * n / 2`; Z3, Coq, Lean and Python disagreed on division, float arrays and Booleans; valid proofs produced unrunnable programs; the relaxed termination rule admitted a never-ending loop; one false step was accepted by Z3 once and caught by Coq and the tests.
- **Honest limits:** only 20 of 164 problems are expressible; the untuned model rarely finds array loops; one incident is unexplained; the model server went down at about 09:10 on 2026-10-05.

Scope and baseline already presented

This report covers work after the square-root-with-Z3 stage, which was already presented; that stage is summarised here only so the later numbers make sense.

- **Paper:** Refine4LLM (Cai, Hóu et al., 2024) lets an LLM choose refinement laws and parameters while a prover checks each step; its Table 6 reports 150/164 HumanEval programs passing both test suites, against 145/128 for GPT-4 alone.
- **Presented baseline:** a general prompt and engine fixes took the square-root example from 7/10 to 99/99 (v6, Z3 only, relaxed termination rule, leaf assist on, 30 calls per trial, `gpt-oss:120b` at low reasoning effort); the 12-case batch had ten cases at 20/20, integer division at 1/20 and the impossible control at 0/5.
- **Fixes behind that baseline:** feedback addressed to the right step, a missing-fact diagnosis (`x >= 0`), feasibility of every sub-specification, a vocabulary check, refusal of splits on arbitrary variables.
- **Ablations measured with that code:** strict termination rule 46/48; leaf assist off 8/30.

Where things stood after the Z3 stage

The search worked on one example and everything else was still open: no independent prover, no dataset results, and several suspected bugs.

Goal you set	State at handover	Work done afterwards
Reliable square root by refinement	solved with Z3 (99/99)	re-measured with the final code: 100/100
Automated Lean and Coq	installed, no backend code	backends, certification, batching, fallback
Exact results on the paper's datasets	HumanEval runner untested	census, baselines, 20 checked specifications, refinement pilot
Same type of results as the paper	tables 6, 7, 9 not reproduced	analogues built and measured

Open items carried forward: the termination-rule default (decision D1); whether the assist stays optional; the Fig. 2 invariant lacking `x >= 0` (question Q1); the HumanEval formal specifications and law library (question Q2); a suspected bug that the Python emitted for derived programs used `/` for whole-number division.

Environment: Windows 11, 16 cores, 23 GB RAM; hosted model behind an ngrok tunnel serving one request at a time; Rocq Platform 2026.07.0 and Lean 4 with Mathlib installed under the user profile; Z3 5.1.0; pytest suite of 320 tests at that point.

Automated Coq and Lean certification

Z3 stays the search prover because it is fast and returns counterexamples; Coq and Lean re-check each finished derivation as independent kernels.

What was built

- Backends `z3`, `coq`, `lean`, cross-checking variants, and fallback backends `z3+lean` and `z3+coq`, where the proof assistant is asked only when Z3 answers *unknown* and only a kernel-checked proof is accepted; Lean never overrules a Z3 counterexample.
- Translators from the specification language to Coq (`coq_translator.py`) and Lean (`lean_translator.py`); Booleans become propositions so the true/false HumanEval problems can be certified.
- A certification pass after the search (`certify_derivation`) and a one-command CLI: `main.py --case C5-sqrt-paper --assist --certify coq lean`.
- Lean batching: all obligations of a derivation in one process, about 10 s instead of about 10 s each (almost all of it loading Mathlib).
- A process-tree killer, because a timed-out `coqc` left `rocqworker` children running and six orphans had burned CPU for half an hour.
- A fix for a Coq translation gap: a bare numeral no longer forces an integer sort, and `subst` was added as a fallback tactic.

What each prover could certify

Test	Coq	Lean
Classic derivations (Factorial, Exponentiation, Linear Search, Max Element), 21 obligations per calculus	9 of 21 (CoqHammer `sauto` added 2)	21 of 21
`choose_num`	0 of 4	4 of 4
Square-root derivation (3 obligations)	3 of 3, about 1.7–2.7 s	about 10–15 s
All 41 test-passing HumanEval derivations	41 of 41	not run at scale

Reading it: Lean's `grind` is stronger than Coq's standard tactics on quantified and array statements; Coq's `UNKNOWN` verdicts mean the tactics gave up, never that something was refuted.

Trust rule adopted: a derivation counts as verified only when an independent kernel has certified it.

Datasets: what can honestly be reproduced

An exact reproduction of the paper's Table 6 is impossible without its HumanEval formal specifications, fine-tuned model and law library, and a model is not deterministic; what can be made exact is everything after the model's answers.

Census of the 164 HumanEval problems by what the language needs

Class	Problems	Needs
A: scalar result, numeric inputs	53	expressible today
B: list result, numeric inputs	28	array writes
C: strings involved	68	a string type
D: dicts, tuples, nested lists	15	more data structures

Direct-model baseline (no refinement, temperature 0, one sample per problem)

System	Original tests	EvalPlus
Paper: GPT-4 alone	145	128
Paper: Refine4LLM	150	150
gpt-oss:120b (ours)	156	147
qwen3-coder-next (ours)	154	147

A bug that first scored 0 of 164: the endpoint forces a JSON reply (`{"code": "python\n..."}`) and the extractor only knew Markdown fences, so it ran the JSON text as Python and failed every problem. `extract_code` now opens JSON replies and drops a stray `python` line; the stored answers were re-scored offline after confirming the harness accepts 164/164 reference solutions.

A bug in emitted programs: the Python for a derived program used `/` for every division, but the prover reads `/` on two integers as whole-number division, so a proved evenness test `n = 2 * (n / 2)` was always true when run. `typed_python.py` now emits `//`

where both sides are integers, and the harness and the old benchmark runner use it.

Checked specifications. The first 12 problems, then 8 more on 2026-10-04 (`add`, `triangle_area`, `multiply`, `x_or_y`, `monotonic`, `can_arrange`, `pairs_sum_to_zero`, `triples_sum_to_zero`). Two mechanical checks replace trust by inspection:

- **Correctness:** for every enumerable HumanEval and EvalPlus input the reference output satisfies the specification (83 to 1011 inputs per problem; ranges too large to enumerate are reported as unchecked, e.g. 581 of `gcd`'s).
- **Strength:** each reference output is replaced by a wrong one (Boolean flipped, number ±1 or doubled) and must be rejected: 590 of 590 rejected.

The checks caught mistakes of mine before any model run: negation `~` binds weakest in the text grammar; Python's `% 10` is not "last digit" for negative numbers (the reference uses `int(str(a)[-1])`); EvalPlus calls `x_or_y` with `n <= 0`; and `triangle_area(5, 3)` must be 7.5, which exposed an evaluator bug (it chose division from runtime values, not declared types).

Hand derivations (no model): 17 of the 20 problems have a derivation the engine accepts law by law; its program passes every original and EvalPlus test except three that are correct but too slow for EvalPlus's largest inputs (`gcd`, `is_prime`, `iscube`). `has_close_elements`, `pairs_sum_to_zero`, `triples_sum_to_zero` need nested loops and were not derived by hand.

Table 7 analogue (scripted derivations, no model): with the extended law `initialised_iteration` every classic program needs exactly one step fewer than with core laws (e.g. Factorial 8 to 7, Max Element 10 to 9); Logarithm, Dutch Flag Sort and Pattern Matching are stated but not derived.

The first model-driven pilot and the 12-case batch (old code)

Before the night's changes, two runs on your laptop gave the evidence that decided what to fix.

12-case batch, 20 trials each (relaxed rule, assist on, Coq on, 30 calls)

Case	Result	Note
A1, A2, A3, B1, B2, B3	20/20 each	2 calls median (B3: 4)

Case	Result	Note
C1, C2, C3, C4	20/20 each	C1, C3, C4 have trivially satisfiable specifications (`x = 0`, `i = N`) so say less
D1 integer division	1/20	19 trials used all 30 calls
A4 impossible (control)	0/5	no soundness alert

Division failed because the model chose the right loop but its set-up and body each needed two assignments in a row; the assist closed only one-line goals. The assist gained equation solving (`q*d + r = n` gives `r := n - q*d`) and a two-assignment search.

First HumanEval refinement pilot (10 problems, 29 runs): 13 derived a program, all 13 passing every test; tier A 7/9, tier B 6/14, tier C 0/6. The model alone got 11 of those 12 right.

Reading the failures of that pilot (the diagnosis that drove v8)

- `is_equal_to_sum_even` 1/3: the model was told the postcondition contained `n = 2 * n / 2`, which reads as always true.
- `choose_num` 0/5, `max_element` 0/2, `gcd`, `is_prime`, `largest_divisor` 0/2 each: array and quantifier syntax errors, loops it could not finish, all 30 calls used.
- `below_threshold` 1/2: its one success set a Boolean with `b := 1` and `b := 0`, which Z3 accepted because z3py coerces Booleans to integers.

Z3 luck: a primality loop was rejected after 10 s on most runs and proved in milliseconds on others with byte-identical input (7 of 12 random seeds proved it in under 10 ms, 5 timed out). The prover now retries a timed-out obligation in up to four short runs from scratch with different seeds, then one patient run; a proof is a proof whatever the seed, so nothing becomes unsound.

The night of 4–5 October: what was changed and why (code v8)

Each change below came from reading a failed or suspicious transcript, not from tuning the prompt; all are general engine or display rules with tests.

Working method. The main folder was in use by your batch, so the code was copied to `Symbolic_LLM_dev` and changed only there; studies ran from the copy and code and results were copied back afterwards. The new code is called v8 because the notes already used "v7" for the evening's division fix.

#	Problem found	Fix	Test
1	**Display bug.** `2 * (n / 2)` ("n is even") was shown to the model as `2 * n / 2`; the renderer dropped parentheses that matter under whole-number division	renderer keeps parentheses for a right operand of `*` that contains a division	300 random formulas are rendered, parsed back and must evaluate identically
2	Quantifier spellings (`forall k:int (...)`, `exists k, ...`) and implication arrows failed with raw parser errors	binder spellings normalised; an implication arrow gets a one-line explanation (`!(A)	
3	A guard with a quantifier was proved and then crashed when turned into Python	guards must be quantifier-free	`test_executable_guards.py`
4	`b := 0` for a Boolean `b` was proved (z3py coerces false to 0)	assignments are type-checked; the assist proposes `true` / `false` for Booleans and `l[i]` reads for numbers	`test_value_kinds.py`
5	The assist's two-assignment search stopped after 4 s, so its outcome depended on machine load	limited by prover queries (cap 1000; division needs 221), 20 s safety net	clock-independence test
6	Z3 divided integers the Euclidean way (7 / -2 = -3) while Coq, Lean and Python round down (-4)	floor division in Z3; divisibility tests `x = k * (x / k)` keep native division (rounding-free, and the case split had made quantified searches time out)	`test_z3_semantics.py`

#	Problem found	Fix	Test
7	`array float` was modelled as `Array(Int, Int)`, so a step valid only over integers was accepted for floats	element sort taken from the declared type	same file
8	Array index had to be a single name (`l[k + 1]` was a syntax error)	index and slice bounds may be arithmetic terms	same file
9	The evaluator chose division from runtime values (`triangle_area(5, 3)` gave 7)	declared `float` names, and their earlier values `x0`, are converted to float	`test_evaluator.py`

Corrected diagnosis (recorded because I first said otherwise). The batch's integer-division failures did *not* come from the 4-second limit: that division run started at 19:25 and the two-assignment search was added at 20:00, so the runs simply did not have it. Shrinking the limit to 0.2 s (a machine about twenty times slower) is a simulated hazard that does reproduce a failure, which is why fix 5 was still worth making.

Specifications: 12 grew to 20 (see Datasets), with hand derivations for 6 of the 8 new ones, so 17 of 20 are known derivable.

Reasoning about what not to do: no prompt change was made after the pilot failures (for example an array-scan example for the loop recipe), because the eight problems added after the fixes serve as an untouched test set and tuning on them would turn it into a training set; the professor's earlier critique was precisely about problem-specific prompts.

How the changes were verified before any model run

No change was measured with the model until it had passed tests, an independent review and offline replays of stored runs.

- **Test suite:** 320 at the start of this phase, 444 after the night's changes (Coq and Lean included); every new rule has a test that fails on the old behaviour.

- **Offline replays:** the stored model replies of past trials are fed back through the new code with no network (`replay.py`). The 99 square-root trials replayed identically; of the 225 stored batch trials 205 replayed identically and the other 20 were exactly the division trials, which now all succeed in 2 to 6 calls; the five impossible-control trials still fail.
- **Independent review:** a second agent that had not seen my reasoning read every change. It re-checked the division encoding against Python for every sign combination, ran 9,000 random formulas through render and parse-back with no mismatch, and found no false refusals in the new type check. It found three small problems, all fixed and tested: binders of type `char` or `array` were mangled by the rewrite; a Boolean defined by an equation lost one assist candidate; the earlier value `x0` of a float was not treated as a float.
- **Cross-checks of the specifications:** `he_specs` re-run after each evaluator change (20 of 20 agree, 590 of 590 mutants rejected).
- **Later, a second independent session** re-checked the headline numbers from raw result rows without a model; its corrections are listed under Incidents.

Final results (v8, 2026-10-05)

All runs use `gpt-oss:120b` at low reasoning effort unless stated, relaxed termination rule, assist on, Coq certification, 30 calls per trial; intervals are Wilson 95%.

Study	Successes	95% interval	Notes
Square root, set 1	50/50	93–100%	median 2 calls, max 4, all Coq-certified
Square root, set 2 (parallel launcher, same code)	50/50	93–100%	the two distinct programs pass 212 concrete inputs
Square root, medium effort	20/20	84–100%	32 s per trial
11 other solvable problems, run 1	220/220	84–100% each	integer division 20/20, all Coq-certified
11 other solvable problems, run 2	220/220	84–100% each	Coq certified 218 (nothing refuted)
Impossible control, both runs	0/5, 0/5	—	sound

Study	Successes	95% interval	Notes
Strict termination rule (v6 code)	46/48	86–99%	median 3 calls, p = 0.11 vs relaxed
Assist off (v6 code)	8/30	14–44%	loop law proposed a median of 12 times in a failed trial

Program check on the 220 derived batch programs: all correct except one division derivation that reads `q` before assigning it (legal in the calculus, a `NameError` in Python).

HumanEval refinement, 121 valid runs (each problem 4 to 8 times; 5 runs cut off by the server outage excluded)

Tier	Runs deriving a program	Notes
A: one assignment or case split (6 problems)	35/42	`car_race_collision` 6/6, `triangle_area` 8/8, `add` 8/8, `is_equal_to_sum_even` 6/6, `right_angle_triangle` 6/6, `multiply` 1/8
B: a loop or chain of cases (5)	7/29	`sum_to_n` 6/6; `max_element`, `below_threshold`, `choose_num`, `can_arrange` none usable
C: searches with nested quantifiers (9)	11/50	none passes the tests
All	**53/121**	**41 pass every original and EvalPlus test, all 41 fully Coq-certified; 7 of 20 problems**

- Coq fully certifies 45 of the 53 derived programs: the 41 plus four valid but unrunnable ones.
- **A real win over the model alone:** `multiply`, which gpt-oss fails on EvalPlus, was derived once (25 calls) as a verified program passing every test.
- Old code on the original 12 problems: 13 of 29 runs; v8 lifts `is_equal_to_sum_even` from 1/3 to 6/6.

- The 12 derived programs that fail a test are all explained: 8 assign a quantified formula (no program can run it), 1 tests the proof-only name `V0`, 1 (`gcd`) is correct but too slow for EvalPlus, 1 is the wrong step described under Incidents, 1 is the never-ending loop described there.
- **Held-out problems** (specified after the v8 fixes): `add` and `triangle_area` every time, `multiply` once, two others derived once each only as unrunnable quantified assignments, three never.

Second model: `qwen3-coder-next` alone passes 154/164 and 147/164; its refinement arms never ran (server outage).

What the HumanEval failures taught, and v9

Every failed or suspicious run was read; hand derivations exist for all problems below except those noted, so these are failures to *find* a derivation, not limits of the engine.

Problem	What happened	Class
`choose_num`	insisted on the test "an even number exists in [x, y]" (existential guard proposed 7 times in one run, 5 in another, refused as unrunnable); twice wrote a correct quantifier-free test but did not take the next split on the parity of `y`	model search
`multiply`	re-split the `else` branch on its parent's test again and again; each new `then` branch has a contradictory precondition and was closed by `skip` (accepted 10 times), so nothing progressed	no check for already-decided tests
`max_element`, `below_threshold`	quantifier scope errors (`… && forall (k:int) A	

Problem	What happened	Class
`largest_divisor`	proved, but the loop body split on `r = V0`, a proof-only name, so the program raised `NameError` on every test; the assist does find the right body `r := r - 1` offline (after about 36 s) but the same query alone is *unknown* even with 10 s, so in the live run it missed	proof/execution gap, Z3 luck
`iscube` and others	`t := (exists k. k*k*k = a)` assigned a quantified formula	proof/execution gap
`gcd`	one run's false step, one never-ending loop and one correct but too slow program; see Incidents. (The `q` start-value issue is division's, see v9 item 5.)	termination, unexplained
`has_close_elements`, `pairs_sum_to_zero`, `triples_sum_to_zero`	nested loops over pairs and triples; no hand derivation either	beyond current search

v9 (separate copy `Symbolic_LLM_v9`; unit-tested, 90 of 90 stored loop derivations replay identically, not yet measured with the model)

1. A test or assigned value may not mention a proof-only name (`V0`, `V1`, `x0`).
2. A quantified value is refused like a quantified test.
3. A split whose test the precondition already decides is refused (one branch could never run).
4. Array values in counterexamples are shown element by element (`[0] = 2438, every other element 2437`).
5. The emitted program gives every frame variable whose start value the specification leaves arbitrary a start value (`q = 0; r = 0`), so a proved program never reads an unbound name.
6. The relaxed termination rule also requires `I and G implies 0 <= V` (see Incidents).

Replaying the stored `largest_divisor` run through v9 succeeds in 2 calls with a runnable program. Future work noted but not built: quantifier elimination (Z3's `qe`) to *offer* the quantifier-free equivalent of a refused guard; seed restarts for the assist's most plausible candidates.

Incidents and how they were handled

1. One false step accepted by Z3, caught by Coq and the tests (unexplained). In one `gcd` run (`he_refine_v7.jsonl`, row 21) the assist closed a sub-problem with `x := g * k`; the obligation is false (a = b = 4, g = 1 satisfies the assumptions and the gcd postcondition fails), so Z3 must have answered "proved".

- Not reproduced: 41 seeds, 1,698 repeats with random time limits on the laptop, four complete re-runs of the assist's search (about 3,700 queries, the only "proved" answers had contradictory assumptions), a replay of the stored row, and a replay of rows 0 to 20 before it in one process (those 21 rows replay exactly, including one that fails when replayed alone, so Z3's answers do depend on process history); a review session's Linux build gave more than 1,500 runs with no wrong answer.
- Containment: Coq refused to certify it (0 of 2 obligations) and the program failed the tests. Every counted success is Coq-certified.
- Status: cause unknown (timing under load — two pilots shared the machine and memory was short — or a solver fault); `probe_wrong_step.py` keeps the repro tooling.

2. A gap in the relaxed termination rule. All pilot runs used the paper-style rule (measure only has to decrease, `V < V0`). A `gcd` derivation was accepted with `while g is not a common divisor: g := g + 1` and measure `a - g`; the measure decreases every time, but the program never stops when a does not divide b. Over naturals `V < V0` suffices, over integers it does not.

- Check: for all 265 loops the model proposed in counted successes, the invariant and the loop test imply the measure is at least 0 (Z3), so no reported success is affected; the one unbounded proposal was a refused step in a square-root run that then succeeded with a bounded measure, and the non-terminating `gcd` program timed out the tests, so it is not a counted success.
- Fix (v9): add the obligation "while the loop runs, the measure is not negative" (`I and G implies 0 <= V`); sound for integers, refuses the `gcd` loop, still accepts the paper's own measure `N - (x+e)^2`, which is negative only after the last iteration.

3. Model server outage from about 09:10 on 2026-10-05. The ngrok tunnel reached its monthly bandwidth limit and answered every call with `HTTP 403`. Runs after that never reached a model (second model's square root 20, v8 without assist 30, a no-assist repeat, last 5 pilot runs) but had been filed as failures. `transport.py` now classifies this cut-off as network-lost, with a test, and all summaries exclude them. The live demo cannot run until the limit resets; an offline replay demo is provided.

4. A launcher I thought I had stopped kept running. Stopping a background job ends only its wrapper shell, not the script, so a second queue ran in parallel with the first from 00:46 until morning on the same v8 code (source hash `8ae219421b4fd444`, unchanged since 23:08). Its files (`v7_sqrt_assist_*`, `v7cases_*`, `he_refine_v7.jsonl`) are extra v8 samples and are counted as such; they have nothing to do with the earlier evening's "v7". Side effect: two pilots shared the model, so wall times are not comparable with sequential runs.

5. Memory pressure. The system's low-memory guard stopped background job wrappers several times (a game was using about 6 GB); the study scripts themselves kept running except two concurrent jobs, which were resumed or covered by other runs.

6. Corrections applied after an independent re-check. Coq certified 218, not 220, of the second batch's division derivations; "the other five held-out problems never" became "never with a runnable program"; the count of unrunnable derivations became nine; "agrees on every test input" became "every *enumerable* test input"; `pilot_compare.py` now reads all three pilot files by default.

7. A wrong diagnosis corrected (batch division failures, described in the night's section).

Our results next to the paper's

The paper's qualitative claims reproduce; its headline count does not, and the reasons are specific.

Paper	Ours	Same direction?
Table 6: GPT-4 alone 145/128; Refine4LLM 150/150	models alone 156/147 and 154/147; refinement verified-and-passing in 41 of 121 runs on 7 of 20 expressible problems	baselines stronger than GPT-4; no EvalPlus drop for derived programs that run (one correct program is too slow); coverage far below 150

Paper	Ours	Same direction?
Table 7: extended laws need fewer steps than core laws	one step fewer in all four classic programs; all obligations certified by Lean (21/21), Coq 9/21	yes, smaller effect (the paper also uses learned laws)
Table 8: law library	not reproduced, no learned library	not comparable
Table 9: ablation	ablations of our components: assist on/off, strict vs relaxed rule, reasoning effort, old vs fixed code	same kind of table, different components
Fig. 2: square root derived by refinement	100/100 with v8; the exact Fig. 2 invariant is rejected because it lacks `x >= 0` (question Q1)	yes, with a finding
Failure analysis: 14 of 164 failures (8 too complex to split, 3 prover too weak, 3 data structures)	failures: model search, display, syntax, proof-versus-execution gaps, one unexplained solver event	different taxonomy, similar spirit

Why "same results" is not reachable: the paper's HumanEval formal specifications, fine-tuned GPT-4 and law library are not public; only 20 of 164 problems are expressible in the current language (strings, list results and built-ins such as `sum` and `len` are a language extension); and the open models alone already exceed the baseline the paper set out to improve, so refinement's value today is the guarantee, not the pass rate.

Contributions and the evidence for each

Claim	Evidence
A reliability protocol: many trials, Wilson intervals, every model reply stored and replayable offline	`replay.py` reproduces 99 of 99 old and 100 of 100 v8 square-root trials; 205 of 225 batch trials (the other 20 are the division trials that now succeed)

Claim	Evidence
Faithful presentation is a correctness property: what the model reads must mean what the prover checks	the `2 * (n / 2)` display bug, fixed with a 300-formula round-trip test; `is_equal_to_sum_even` 1/3 to 6/6
The search prover, certifiers and executed program must agree on meaning	division rounding, float arrays, Booleans aligned and tested; reviewer checked division against Python for every sign combination
A proof is not a program: running derived programs exposes valid proofs that cannot run	nine unrunnable derivations (quantified values, proof-only `V0`), a read-before-assign start value, a non-terminating loop
Defence in depth works	a step Z3 accepted wrongly was refused by Coq and caught by the tests
A termination rule that is sound for integers and admits the paper's measure	`I and G implies 0 <= V` added to the relaxed rule; paper measure `N - (x+e)^2` accepted, `gcd` loop refused
Checked specifications	reference outputs on every enumerable test input plus 590 of 590 mutants rejected; caught three of my own errors before any run
Updated baseline	two open models alone pass 156/147 and 154/147

The general lesson recorded in the notes: the fixes that mattered were about what the engine checks or says after a failure, not about the prompt's wording.

Limitations and what not to claim

- Not "we reproduced 150 of 164": only 20 problems are expressible, and refinement solves 7 of them.
- Not "refinement beats the model on tests": the models alone are very strong; gpt-oss fails 2 of these 20 and refinement fixed one (`multiply`) once in 8 runs.
- Not "Z3 alone is enough", and not "relaxed-rule derivations always terminate": certification, bounded measures and running the programs are what make counted successes trustworthy.
- Not "fast": three derived programs are proved correct but too slow for EvalPlus's largest inputs; specifications say nothing about running time.

- One model family for refinement, mostly low reasoning effort; the second model and the v8 assist-off arm were not measured (outage).
- The specifications are mine, checked but not the paper's; 581 of `gcd`'s inputs were too large to enumerate.
- The v9 changes are unit-tested and replay-checked but not measured with the model.
- One incident (the false step) is unexplained, and wall-clock times of the night's runs are not comparable because two launchers shared the model.

Open questions and decisions needed

For Dr. Hou (file `18_Questions_for_Dr_Hou.md`, five questions, ready to forward once your name is added)

1. Fig. 2's invariant `x*x <= N` lacks `x >= 0`; with `x = -3, e = 1, N = 10` the loop body fails. Is the figure abbreviating, is only partial correctness checked, or is Lemma 6.7's variant condition weaker?
2. Can he share the HumanEval formal specifications (adapted from BigCode), the 164 prompts and the law library?
3. How do `L_spec` and `L_pl` express list results, strings and built-ins such as `sum`, `len`, `%`?
4. How was Table 6 run: one derivation or best of several, temperature, call budget, what counts as a failure for the fall-back?
5. Three semantic choices: is integer division floor (Coq's `Z.div`); may a guard contain a quantifier; are Booleans propositions and `b := 0` ill-typed? Added: are loop measures natural numbers?

Decisions for you

- Termination default: strict (`0 <= V < V0`, one more model call per run, 46/48 vs 99/99, not significantly different) or relaxed plus "measure not negative while the loop runs" (sound, admits the paper's measure).
- Whether the leaf assist is on by default (it is the largest single factor: 8/30 without it) or stays an optional arm.
- Whether to measure v9 with the model once the server limit resets, and whether to extend the language (arrays writes, strings) to widen coverage.

Reproducibility: where everything is

Every result file stores the model's raw replies, so any trial can be re-derived offline; everything after the model is exactly reproducible, the model's answers are not.

What	Location or command
Showcase, paper draft, talk outline	`Reading Material/19`, `20`, `21`; LaTeX `paper/main.tex`, figures `paper/fig_*.png`, redraw with `python paper/make_figures.py`
Square-root and batch studies	`benchmarks/results/reliability/` (`v8_*`, `v7_*20261005*`, `v8cases_*`, `v7cases_*`); table: `python benchmarks/summarize_reliability.py "<glob>" --markdown`
HumanEval pilot rows	`benchmarks/results/datasets/he_refine_v8.jsonl`, `he_refine_v8_heldout.jsonl`, `he_refine_v7.jsonl` (all v8 code); table: `python benchmarks/pilot_compare.py`
Baselines	`direct_baseline_gpt-oss-120b-low_rescored.jsonl`, `direct_baseline_qwen3-coder-next.jsonl`
Re-check specifications	`python -m benchmarks.datasets.he_specs` (20 of 20 agree, 590 of 590 mutants rejected)
Hand derivations	`python -m benchmarks.datasets.he_oracle --show`
Replay a stored study offline	`python benchmarks/replay.py <file.jsonl> --assist --strict 0 --certify coq`
Offline demo (model server down)	`python benchmarks/replay.py "benchmarks/results/reliability/v6_relaxed_assist_*.jsonl" --rows 0 1 --assist --strict 0 --certify coq`
Live demo (once the limit resets)	`python main.py --case C5-sqrt-paper --config gpt-oss-120b-low --assist --certify coq lean`
Test suite	`python -m pytest FormalLLM/tests -q` (about 3 minutes with Coq and Lean)

What	Location or command
Repro tooling for the unexplained step	`python benchmarks/probe_wrong_step.py fuzz 1200 7`

Folders: main repo `Symbolic_LLM` holds the v8 code and all results; `Symbolic_LLM_dev` is the copy the studies ran from (code frozen, hash `8ae219421b4fd444`); `Symbolic_LLM_v9` holds the unmeasured v9 fixes. The model login lives outside the repository in `~/.formalllm/remote_auth`. Nothing is committed.

Timeline (IST, newest first)

Date and time	Event
2026-10-05, about 12:45	Reproduction probes for the false step finish: 1,698 repeats and a replay of rows 0–20, no wrong "proved"; incident stays unexplained
2026-10-05, late morning	Outage handled: `transport.py` classifies the HTTP 403 cut-off as network-lost; final tables rebuilt over 121 valid pilot runs; termination gap found, checked on 265 loops and repaired in v9; review-session corrections applied; docs 19, 20, 21, 10, 14, 18 and `main.tex` finalised
2026-10-05, about 09:10	Model tunnel reaches its monthly bandwidth limit; every later call returns HTTP 403
2026-10-05, 06:24–09:27	Medium effort 20/20; second model's baseline 154/147; second batch and pilot pass 2 finish; later arms cut off
2026-10-05, 00:46–06:24	v8 runs: square root 50/50 twice, pilot passes, 12-case batch 220/220 twice (second launcher in parallel from 00:46)
2026-10-04, 23:08	v8 code frozen after a clean 444-test run, replays and an independent review (hash `8ae219421b4fd444`)

Date and time	Event
2026-10-04, 22:00–23:08	Display bug, quantifier syntax, executable guards, typed assignments, query-limited pair search, Z3 floor division, float arrays, array-index terms, 8 new specifications; three review findings fixed
2026-10-04, 19:00–22:00	12-case batch and first model pilot on old code (13 of 29; 1 of 20 division) read for failure diagnosis; a low-memory stop of a background wrapper (the queue kept running)
2026-10-04, earlier	Cloud session: `transport.py`, `cross_check.py`, `reliability_batch.py`, baseline re-scoring (0 of 164 bug fixed: 156/147), `he_oracle.py` hand derivations, Z3 restarts
2026-10-03, 11:22	Background chain of ablation arms dies after the first two steps (square root 99/99 and the HumanEval baseline); remaining arms later re-queued
2026-10-03, morning	Coq and Lean backends, certification, batching, process-tree killer, CLI; HumanEval runner, evaluator, typed Python emitter; 320 tests